



Working with Floating-Point Numbers

Practical Skills Module 2: Topic 3

[Visit our website](#)

Practical Skills Module 2: Topic 3

Topic Code	PM-02-PS03:
Topic	Create a float in Python

Scope of Practical Skill

Given an applicable instruction and access to a learning platform, the learner must be able to:

- PA0301 Create a float
- PA0302 Round floats
- PA0303 Convert a list of strings to a float
- PA0304 Compare floats
- PA0305 Multiply floats
- PA0306 Format floats
- PA0307 Create a list of floats
- PA0308 Add floats
- PA0309 Store a float in an integer
- PA0310 Sum a list of floats
- PA0311 Convert an integer to a float

Applied Knowledge

- AK0301 Concept, definition and functions of Python data types

Internal Assessment Criteria

- IAC0301 Expected results with Python floats are achieved

Resources:

Knowledge	Information from Python Programmer Curriculum
National Curriculum Framework	900221-000-00-PM-02



Practical task

Follow the facilitator's instructions to complete the following activities:

Scenario: Building a "Grading System for Students"

In this task, you are building a **Grading System** using Python. The program will calculate, manipulate, and present students' grades. You'll practise working with **floats**, **lists**, **numeric conversions**, and **basic formatting**, as well as using **input()** to collect a student's grade.

Create a Python file called **grading_system.py** and complete the following:

Step 1: Assign a float to represent a grade.

- Use the `input()` function to get a student's **assignment grade** from the user.
- Convert the input into a float and store it in a variable.

Step 2: Round the float to the nearest integer.

- Round the student's grade to the nearest whole number using Python's `round()` function.
- Print the rounded grade with a label.
 - *Example output:* Rounded grade : 84

Step 3: Convert a list of strings to floats.

- Create a list of **grades stored as strings** (e.g., `["78.5", "82.0", "90.25"]`).
- Convert each element in the list to a float and store them in a new list.

Step 4: Compare floats.

- Set a **passing threshold** (e.g., `50.0`).
- Use an `if` statement to compare the original input grade to the threshold.
- Print a message indicating whether the student passed or failed.

- *Example output:* Passed: True or Student did not meet the threshold.

Step 5: Multiply floats.

- Create a **weight factor** (e.g., 0.4 to represent 40%).
- Multiply the student's grade by the weight factor to get the **weighted score**.
- Print the weighted score.

Step 6: Format a float to two decimal places.

- Format the weighted score to **two decimal places** for clean presentation.
- Print the formatted score with a label.
 - *Example output:* Weighted score: 33.60

Step 7: Create a list of floats.

- Create a list to store at least **three different assignment grades** as float values.
- Print the list to confirm it's correctly formatted.

Step 8: Add floats together.

- Add the grades in the list together to get the **total score**.
- Print the total with a clear label.
 - *Example output:* Total score: 254.75

Step 9: Convert a float to an integer.

- Convert the total score into an integer using the `int()` function (this will round down).
- Print the converted value.
 - *Example output:* Total score rounded down: 254

Step 10: Use `sum()` on a list of floats.

- Use the built-in `sum()` function to calculate the **sum of all grades** in the list created in Step 7.

- Print the result to confirm accuracy.

Step 11: Convert an integer to a float.

- Assume you have an integer value representing the number of assignments (e.g., 3).
- Convert this integer to a float using `float()`.
- Print the converted value.
 - *Example output:* `Number of assignments as float: 3.0`



Take note

Important: Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.
